

FinanSync — Proyecto Final (Web)

Documentación del proyecto · Curso Full Stack Web & Mobile con Integración de IA · Mayo 2026

Descripción general

Esta entrega corresponde al **Paso 7 (Proyecto Final)** del curso. La aplicación web está **completamente funcional**: todas las pantallas operan con datos reales desde Cloud Firestore, la autenticación con Firebase cubre registro, login con email/contraseña y con Google, y la integración con Gemini 2.5 Flash ofrece dos flujos de inteligencia artificial — perspectivas automáticas y chat conversacional personalizado. Se añaden conversión de monedas en tiempo real, modo oscuro con persistencia y gestión completa de preferencias de usuario.

Todos los datos son reales y persisten en Cloud Firestore. La estructura de la base de datos está unificada con la aplicación Android para garantizar compatibilidad entre plataformas.

Avances respecto a Pre-delivery (Beta)

Área	Pre-delivery	Proyecto Final
Chat IA	No disponible	Nuevo Gemini 2.5 Flash con contexto financiero real
Conversión de monedas	No disponible	Nuevo Soporte MXN, USD, EUR, GTQ, COP, ARS en tiempo real
Modo oscuro	No disponible	Nuevo Tema claro/oscuro persistido en localStorage
Configuración	UI solo	Operativo 5 pestañas funcionales conectadas a Firebase
Notificaciones	No disponible	Nuevo Preferencias por tipo, persistidas en Firestore
Dashboard	Funcional	Operativo Con conversión de moneda y gráficas actualizadas
Transacciones	Funcional	Operativo Moneda por transacción + filtros mejorados
Presupuestos	Funcional	Operativo Alertas con control de notificaciones
Metas	Funcional	Operativo Contribuciones con conversión de moneda

Configuración e instalación

Requisitos previos

Herramienta	Versión mínima
Node.js	18+
npm	9+
Proyecto Firebase	Authentication + Firestore habilitados
API Key de Gemini	Google AI Studio — modelo gemini-2.5-flash

Variables de entorno

Crear el archivo `.env.local` en la raíz de `FinalProject/web/` :

```
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=...
NEXT_PUBLIC_FIREBASE_PROJECT_ID=...
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=...
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=...
NEXT_PUBLIC_FIREBASE_APP_ID=...
GEMINI_API_KEY=...
```

`GEMINI_API_KEY` no tiene prefijo `NEXT_PUBLIC_` — solo existe en el servidor (API Routes) y nunca se expone al navegador. El archivo `.env.local` está en `.gitignore` y nunca se sube al repositorio.

Pasos de instalación

```
# Navegar a la carpeta del proyecto
cd FinanSync/FinalProject/web

# Instalar dependencias
npm install

# Iniciar servidor de desarrollo
npm run dev
```

```
# Abrir en el navegador
# http://localhost:3000

# Build de producción
npm run build && npm start
```

Reglas de Firestore

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /transacciones/{doc} {
      allow read, write: if request.auth != null && request.auth.uid == resource.data.uid;
      allow create: if request.auth != null && request.auth.uid == request.resource.data.uid;
    }
    match /presupuestos/{doc} {
      allow read, write: if request.auth != null && request.auth.uid == resource.data.uid;
      allow create: if request.auth != null && request.auth.uid == request.resource.data.uid;
    }
    match /metas/{doc} {
      allow read, write: if request.auth != null && request.auth.uid == resource.data.uid;
      allow create: if request.auth != null && request.auth.uid == request.resource.data.uid;
    }
    match /users/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

Stack tecnológico

Tecnología	Versión	Uso
Next.js	15.2.4	Framework principal — App Router, API Routes serverless
React	18.3.1	Librería de componentes UI y gestión de estado
TypeScript	5.x	Tipado estático en modelos, servicios y componentes
Tailwind CSS	3.4.1	Framework CSS utilitario con soporte modo oscuro
Firebase SDK	12.11.0	Authentication y Cloud Firestore
Recharts	2.15.0	Gráficas financieras (área, pastel, línea, barras)
Lucide React	0.475.0	Íconos

jsDelivr Currency API	pública	Tasas de cambio en tiempo real (caché 24 h)
Gemini 2.5 Flash	REST API	Perspectivas financieras y chat conversacional

Arquitectura del proyecto

Estructura de carpetas

```
FinalProject/web/
├─ app/
│   ├─ layout.tsx           # Layout raíz con todos los Providers
│   ├─ globals.css
│   ├─ page.tsx             # Login (pública)
│   ├─ registrarse/page.tsx # Registro (pública)
│   ├─ api/
│   │   ├─ chat/route.ts    # Proxy Gemini – chat conversacional
│   │   ├─ perspectivas/route.ts # Proxy Gemini – análisis financiero
│   │   └─ tipo-cambio/route.ts # Proxy jsDelivr – tasas de cambio (caché 24h)
│   └─ (app)/               # Rutas protegidas (requieren sesión)
│       ├─ layout.tsx       # Verifica auth → redirige si no hay sesión
│       ├─ dashboard/page.tsx
│       ├─ transacciones/page.tsx
│       ├─ transacciones/agregar/page.tsx
│       ├─ presupuesto/page.tsx
│       ├─ metas/page.tsx
│       ├─ perspectivas-ia/page.tsx
│       ├─ chat-ia/page.tsx
│       └─ configuracion/page.tsx
├─ components/
│   └─ layout/Sidebar.tsx
├─ context/
│   ├─ AuthContext.tsx      # Sesión Firebase – user, loading
│   ├─ FinanzasContext.tsx  # Moneda, categorías, conversión entre divisas
│   ├─ NotifContext.tsx     # Preferencias de notificaciones
│   └─ ThemeContext.tsx     # Tema claro/oscuro con persistencia
└─ lib/
    ├─ firebase.ts          # Inicialización Firebase (Auth + Firestore)
    ├─ transacciones.ts     # CRUD + tipos
    ├─ presupuestos.ts       # CRUD + tipos
    └─ metas.ts             # CRUD + tipos + helpers de fecha
```

Capas de la aplicación

Capa	Archivos	Responsabilidad
------	----------	-----------------

Presentación	<code>app/</code> , <code>components/</code>	Renderizado de UI, manejo de eventos, estado local con <code>useState</code>
Estado global	<code>context/</code>	Sesión, moneda/conversión, notificaciones, tema — compartidos entre páginas
Servicios	<code>lib/transacciones.ts</code> , <code>lib/presupuestos.ts</code> , <code>lib/metast.ts</code>	Funciones async puras de acceso a Firestore — las páginas nunca usan el SDK directamente
API serverless	<code>app/api/</code>	Proxies seguros hacia Gemini y jsDelivr — las claves nunca llegan al navegador
Infraestructura	<code>lib/firebase.ts</code> , <code>context/AuthContext.tsx</code>	Inicialización de Firebase, suscripción al estado de autenticación

Sistema de autenticación

Firebase Authentication

Se implementaron dos métodos de inicio de sesión unificados entre web y Android:

- **Email y contraseña:** registro con `createUserWithEmailAndPassword` , login con `signInWithEmailAndPassword` .
- **Google:** login con `signInWithPopup` y `GoogleAuthProvider` .

AuthContext

`context/AuthContext.tsx` expone el usuario actual mediante `onAuthStateChanged` , que Firebase ejecuta automáticamente al cargar la página para restaurar la sesión, y cada vez que el usuario inicia o cierra sesión. Comparte `user` y `loading` .

Protección de rutas

El layout de `(app)/layout.tsx` consume el `AuthContext` y redirige a `/` si, una vez que `loading` es false, no hay sesión activa.

```
useEffect(() => {
  if (!loading && !user) router.push('/')
}, [user, loading, router])
```

Registro de usuario

```
// 1. Crear usuario en Firebase Auth
const { user } = await createUserWithEmailAndPassword(auth, email, password)

// 2. Guardar displayName en Auth
await updateProfile(user, { displayName: nombre })

// 3. Guardar datos adicionales en Firestore
await setDoc(doc(db, 'users', user.uid), { nombre, email, creadoEn })
```

Seguridad de contraseña

El cambio de contraseña requiere re-autenticación previa con `reauthenticateWithCredential` antes de llamar a `updatePassword`. Los usuarios de Google ven un mensaje informativo redirigiendo a `myaccount.google.com` ya que su contraseña es gestionada por Google.

Estructura de datos en Cloud Firestore

Todas las colecciones se definen a nivel raíz con un campo `uid` para filtrar por usuario. Esta estructura es compatible con la aplicación Android — ambas plataformas leen y escriben en las mismas colecciones.

Colección: `transacciones`

Campo	Tipo	Descripción
uid	string	ID del usuario propietario
descripcion	string	Descripción de la transacción
categoria	string	Categoría (Alimentación, Transporte, etc.)
monto	number	Valor numérico en la moneda de origen
tipo	string	<code>INGRESO</code> o <code>GASTO</code> ; la web convierte a <code>income / expense</code> al leer
monedaOrigen	string	Moneda en la que se registró (GTQ, USD, MXN...)
fecha	Timestamp	Para ordenamiento en queries
fechaISO	string	Formato <code>YYYY-MM-DD</code> para cálculos y filtros en el cliente

Colección: `presupuestos`

Campo	Tipo	Descripción
-------	------	-------------

uid	string	ID del usuario propietario
categoria	string	Categoría asociada
limiteMonthly	number	Límite mensual de gasto
mes	number	Mes numérico (1–12)
anio	number	Año del presupuesto
colorHex	string	Color en hex (ej. <code>#3b82f6</code>)
icono	string	Emoji asociado (ej. 🏠)
monedaOrigen	string	Moneda en la que se definió el límite

Colección: `metas`

Campo	Tipo	Descripción
uid	string	ID del usuario propietario
nombre	string	Nombre de la meta de ahorro
objetivo	number	Monto total a alcanzar
actual	number	Monto ahorrado hasta el momento
fechaLimite	Timestamp	Fecha límite
colorHex	string	Color en hex
icono	string	Emoji de la meta
completada	boolean	True cuando <code>actual >= objetivo</code>
monedaOrigen	string	Moneda en la que se definió la meta

Colección: `users`

Campo	Tipo	Descripción
nombre	string	Nombre del usuario (sincronizado con Firebase Auth)
moneda	string	Moneda preferida para mostrar montos
categorias	string[]	Categorías personalizadas de gasto

notifPresupuesto	boolean	Alertas cuando un presupuesto alcanza $\geq 80\%$
notifMetas	boolean	Actualizaciones de progreso de metas
notifIA	boolean	Recomendaciones automáticas de Perspectivas IA

Compatibilidad Android: el campo `tipo` se almacena como INGRESO/GASTO (formato Android). La capa de servicios web convierte estos valores al leer, sin modificar Firestore. Los Timestamps son compatibles entre ambas plataformas.

Capa de servicios Firestore (lib/)

Cada archivo exporta funciones async puras que encapsulan toda la lógica de acceso a Firestore. Las páginas solo importan estas funciones y nunca interactúan directamente con el SDK de Firebase.

transacciones.ts

Función	Descripción
<code>getTransacciones(uid)</code>	Obtiene todas las transacciones del usuario ordenadas por fecha descendente
<code>agregarTransaccion(uid, data)</code>	Crea transacción con <code>monedaOrigen</code> y conversión de tipo a formato Android
<code>eliminarTransaccion(uid, id)</code>	Elimina una transacción por ID
<code>formatearFecha(isoDate)</code>	Convierte <code>YYYY-MM-DD</code> a formato legible (<i>14 feb 2026</i>)

presupuestos.ts

Función	Descripción
<code>getPresupuestos(uid, mes, anio)</code>	Obtiene presupuestos filtrados por mes y año
<code>agregarPresupuesto(uid, data)</code>	Crea presupuesto con <code>monedaOrigen</code>
<code>eliminarPresupuesto(id)</code>	Elimina un presupuesto por ID

metas.ts

Función	Descripción
---------	-------------

<code>getMetas(uid)</code>	Obtiene todas las metas del usuario
<code>agregarMeta(uid, data)</code>	Crea meta con <code>monedaOrigen</code> y Timestamp de fecha límite
<code>agregarContribucion(id, nuevoActual, objetivo)</code>	Actualiza <code>actual</code> y marca <code>completada</code> si aplica
<code>eliminarMeta(id)</code>	Elimina una meta por ID
<code>diasRestantes(isoDate)</code>	Calcula días que faltan a la fecha límite

Contextos React globales

FinanzasContext

Gestiona la moneda preferida del usuario, las categorías de gasto y la conversión de divisas. Carga las tasas de cambio desde `/api/tipo-cambio` al iniciar sesión y las mantiene en estado durante toda la sesión.

Valor / Función	Descripción
<code>finanzas.moneda</code>	Moneda activa del usuario (GTQ, USD, MXN, EUR, COP, ARS)
<code>finanzas.categorias</code>	Lista de categorías personalizadas
<code>convertir(monto, monedaOrigen)</code>	Convierte un monto a la moneda actual del usuario
<code>convertirEntre(monto, de, a)</code>	Conversión libre entre cualquier par de monedas
<code>formatear(monto, monedaOrigen?)</code>	Convierte y formatea: "Q1,500.00 GTQ"
<code>guardar(vals)</code>	Persiste moneda y categorías en Firestore

NotifContext

Almacena y persiste los tres toggles de notificaciones del usuario (`presupuesto` , `metas` , `ia`). Se carga desde Firestore al iniciar sesión y se guarda al cambiar en Configuración.

ThemeContext

Gestiona el tema claro/oscuro. Al cambiar de tema aplica la clase `dark` al elemento `<html>` raíz (Tailwind dark mode) y persiste la preferencia en `localStorage` para restaurarla en recargas sin necesidad de llamar a Firebase.

Integración con Gemini 2.5 Flash

Todos los llamados a Gemini se realizan desde API Routes de Next.js (servidor), nunca desde el cliente. La clave `GEMINI_API_KEY` solo existe en el proceso de Node.js y nunca llega al navegador.

Ruta `/api/perspectivas` — Análisis automático

Recibe el resumen financiero del mes y genera 5 perspectivas estructuradas. El prompt instruye al modelo a devolver un array JSON con objetos de tipo:

```
{ "titulo": "...", "descripcion": "...", "tipo": "alerta" | "positivo" | "sugerencia", "ahorro": 0 }
```

El campo `ahorro` es opcional y contiene el monto mensual estimado de ahorro cuando aplica. La API route parsea y valida la respuesta antes de devolverla al cliente.

Ruta `/api/chat` — Chat conversacional

Recibe el historial de mensajes y el contexto financiero completo del usuario. Construye un system prompt con:

- Nombre del usuario y moneda activa
- Ingresos y gastos totales del mes, tasa de ahorro
- Desglose de gastos por categoría
- Estado de cada presupuesto (gastado vs límite con porcentaje)
- Progreso y días restantes de cada meta de ahorro

El modelo responde exclusivamente en español con consejos concretos basados en los números reales del usuario.

Flujo de la integración

```
// Perspectivas
1. /perspectivas-ia carga datos desde Firestore
2. Calcula resumen del mes (ingresos, gastos, presupuestos, metas)
3. POST /api/perspectivas con el resumen
4. API route construye prompt → llama a Gemini REST API
5. Parsea JSON → devuelve array de perspectivas
6. La página renderiza 5 tarjetas coloreadas

// Chat
1. /chat-ia carga contexto financiero del mes desde Firestore
2. Usuario escribe mensaje
3. POST /api/chat con { messages, contexto }
4. API route antepone system prompt con datos del usuario
```

- 5. Llama a Gemini con historial completo
- 6. Devuelve respuesta → se muestra en el chat

Sistema de conversión de monedas

El endpoint `/api/tipo-cambio` consulta la API pública de `@fawazahmed0/currency-api` (distribuida vía jsDelivr CDN) y devuelve las tasas de todas las monedas relativas al USD. Next.js cachea la respuesta 24 horas con `revalidate: 86400` para no consultar el API en cada petición.

`FinanzasContext` carga estas tasas al iniciar sesión usando la fórmula: $\text{monto_convertido} = \text{monto} / \text{tasa_origen} \times \text{tasa_destino}$. Todas las páginas llaman a `formatear()` para mostrar montos en la moneda del usuario, sin importar en qué moneda se registró cada dato.

Monedas soportadas: **GTQ, MXN, USD, EUR, COP, ARS**.

Pantallas implementadas

Ruta	Pantalla	Estado	Descripción
<code>/</code>	Login	Funcional	Email/contraseña + Google Sign-In. Redirige al dashboard al autenticarse.
<code>/registrarse</code>	Registro	Funcional	Crea cuenta en Firebase Auth y guarda perfil en Firestore.
<code>/dashboard</code>	Panel principal	Funcional	Saldo total, ingresos/gastos del mes, gráfica de área (6 meses), gráfica de pastel por categoría, últimas transacciones. Todos los montos en la moneda del usuario.
<code>/transacciones</code>	Transacciones	Funcional	Lista con búsqueda por texto, filtro por tipo y categoría. Resumen al pie. Eliminación individual.
<code>/transacciones/agregar</code>	Agregar transacción	Funcional	Formulario que guarda en Firestore con <code>monedaOrigen</code> del momento.
<code>/presupuesto</code>	Presupuesto	Funcional	CRUD de presupuestos mensuales. Progreso calculado cruzando transacciones del mes. Alertas al 80% (si notificaciones activas).

/metas	Metas	Funcional	CRUD de metas de ahorro. Contribuciones con conversión de moneda. Banners progresivos (50%, 75%, 100%).
/perspectivas-ia	Perspectivas IA	Funcional	Health score (0-100), gráficas de tendencia y comparación. 5 perspectivas generadas por Gemini clasificadas por tipo.
/chat-ia	Chat IA	Funcional	Chat con asesor Gemini que conoce todos los datos financieros del mes. Historial de conversación, scroll automático, sugerencias rápidas.
/configuracion	Configuración	Funcional	5 pestañas: Perfil (nombre en Auth), Notificaciones (toggles Firestore), Seguridad (cambio de contraseña), Apariencia (tema claro/oscuro), Finanzas (moneda + categorías).

Decisiones de diseño

Colecciones raíz con campo `uid`

Se eligió estructura de colecciones a nivel raíz con campo `uid` en lugar de subcolecciones bajo cada usuario para mantener compatibilidad exacta con Android, que ya usaba este esquema.

API Routes de Next.js como proxies seguros

Las llamadas a Gemini y a jsDelivr se realizan desde el servidor (`app/api/`). Esto mantiene las claves fuera del bundle del navegador, centraliza el manejo de errores y permite parsear y validar las respuestas antes de enviarlas al cliente.

Contexto API en lugar de Redux

El estado global es pequeño y predecible (sesión, moneda, notificaciones, tema), por lo que React Context API cubre todas las necesidades sin introducir dependencias adicionales como Redux o Zustand.

Conversión de tipos Android ↔ Web

Android almacena el tipo de transacción como `INGRESO` / `GASTO` y las fechas como `Long` en milisegundos. La capa `lib/` convierte estos valores al leer y los serializa de vuelta al formato Android al escribir, sin modificar la estructura de Firestore.

Progreso de presupuesto calculado en el cliente

El gasto real de cada presupuesto se calcula cruzando las transacciones del mes con la categoría del presupuesto, directamente en el cliente. No se guarda un campo `spent` en Firestore para evitar inconsistencias entre plataformas: cada una calcula este valor de forma independiente.

Paleta de colores

Color	Uso
Verde <code>#22c55e</code>	Acciones primarias, ingresos, metas completadas, sidebar activo
Azul <code>#2563eb</code>	Sección IA, botones secundarios, login
Rojo <code>#ef4444</code>	Gastos, presupuestos excedidos, alertas, zona peligrosa
Ámbar <code>#f59e0b</code>	Advertencias de presupuesto ($\geq 80\%$)
Slate <code>#f1f5f9</code>	Fondo general (modo claro)